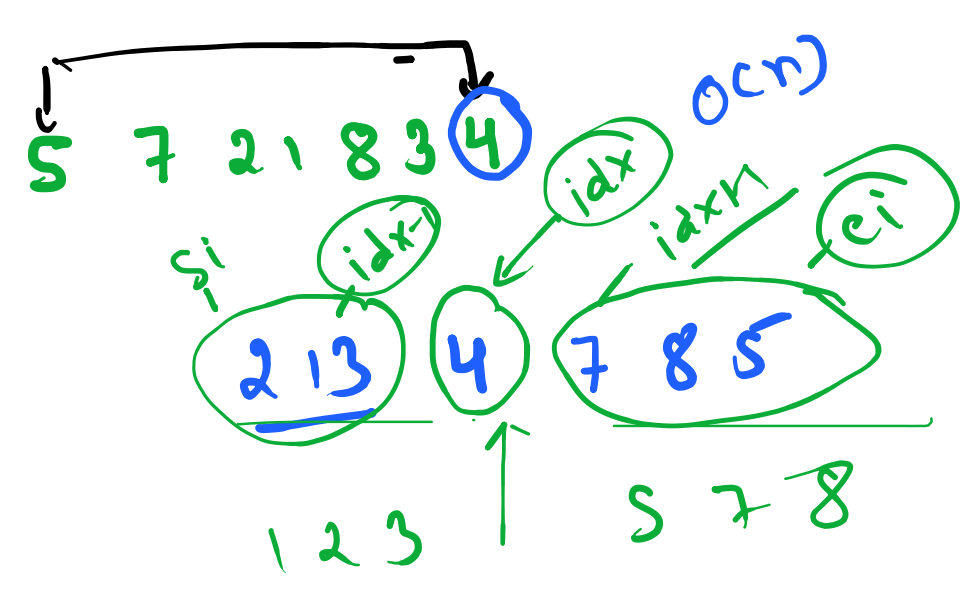
Lec-21

09/05/26 12:52 PM

si
5 7 2 1 8 3 4 ei

① bubble → n2
Slect → n2
Inster → n2
Merge → nlog(n)
Quick → " / n2 → nlog(n)

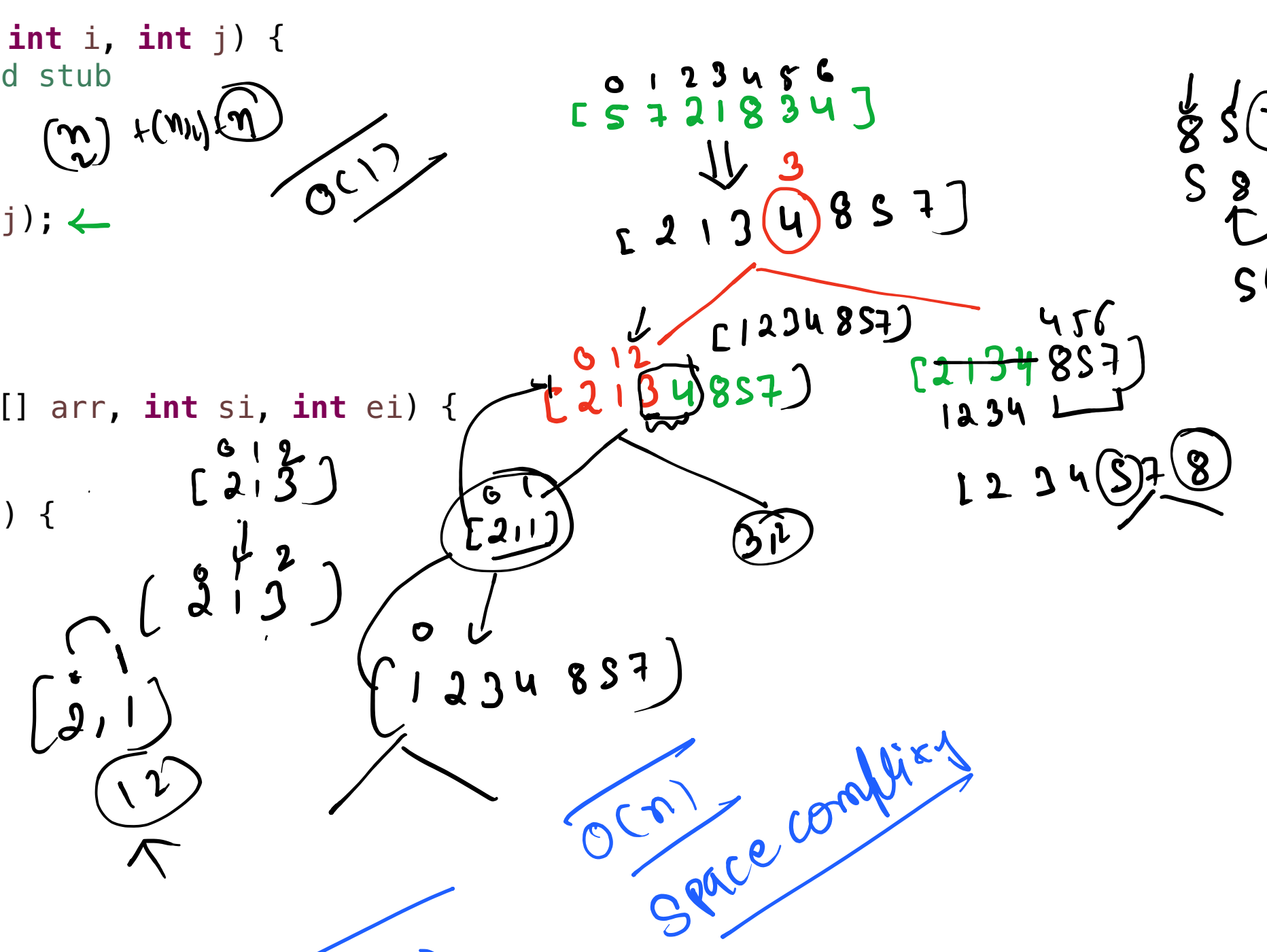


```
public static void Sort(int[] arr, int i, int j) {
    // TODO Auto-generated method stub
    if (i >= j) {
        return;
    }
    int idx = Partition(arr, i, j);
    Sort(arr, i, idx - 1);
    Sort(arr, idx + 1, j);
}

public static int Partition(int[] arr, int si, int ei) {
    int item = arr[ei];
    int idx = si;
    for (int i = si; i < ei; i++) {
        if (arr[i] < item) {
            int temp = arr[i];
            arr[i] = arr[idx];
            arr[idx] = temp;
            idx++;
        }
    }
    int temp = arr[ei];
    arr[ei] = arr[idx];
    arr[idx] = temp;
    return idx;
}
```

$\binom{n}{2} + (n_1) \; (n)$

O(1)



O(n) Space complexity

N log(n) T.C

Quick Sort
Worst Case → when → sorted
N²
nlog(n)

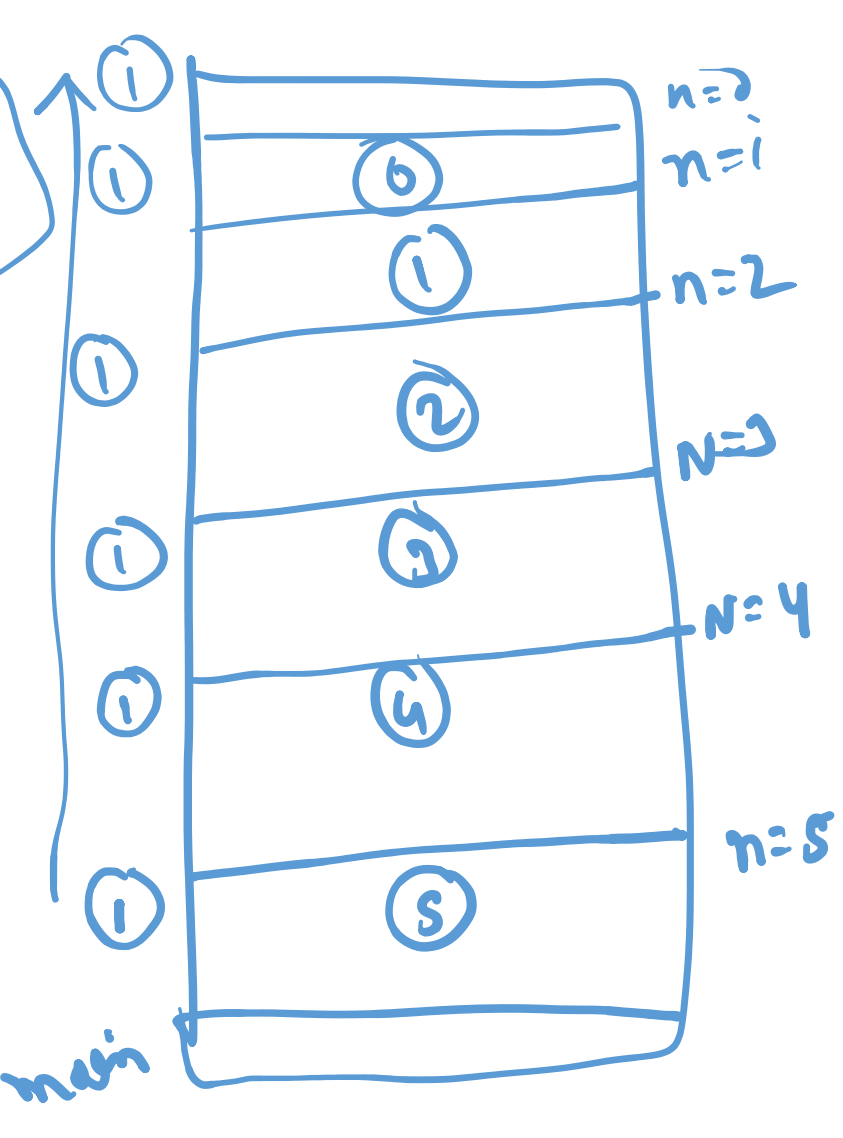
---

Fact
Power
PI
PD
Range

```
public static int fac(int n) {
    //base case
    if(n==0) {
        return 1;
    }
    int f = fac(n - 1);// SP
    return f * n;
}
```

pre i=0 i<n i++ (
3

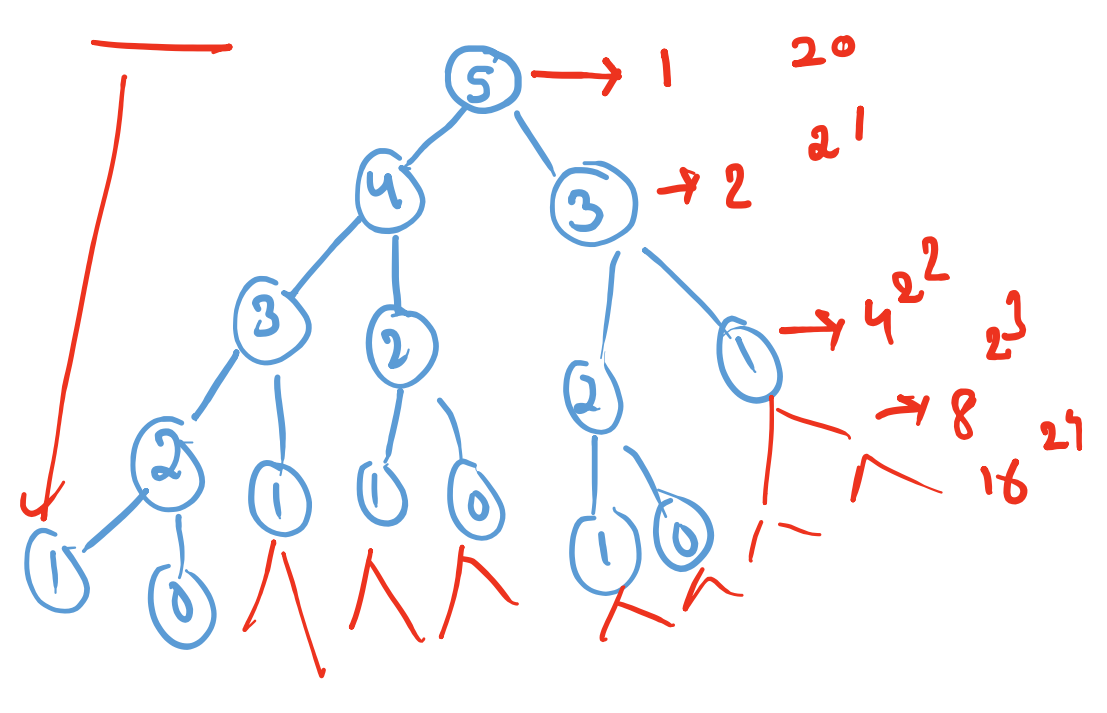
S+u+3+1
n×(n+1)/2
n²
n→ fome
O(1)
O(n)



f(int i, int n) {
if(i>=n) {
return
}
f(i×3, n)
}

```
public static int fib(int n) {
    if (n == 0 || n == 1) {
        return n;
    }
    int f1 = fib(n - 1);
    int f2 = fib(n - 2);
    return f1 + f2;
}
```

Pre i=0 i<n



$2^0 \; 2^1 \; 2^3 \; 2^4 \; \dots \; 2^{N-1}$

$\frac{2^0[2^N - 1]}{2-1} = \frac{1[2^N-1]}{1}$

$= 2^N - 1$

① code

T(n) = n

```
public static int fac(int n) {
    //base case
    if(n==0) {
        return 1;
    }
    int f = fac(n - 1);// SP
    return f * n;
}
```

pow

$T(N) = \cancel{T(N-1)} + 1$

$\cancel{T(N-1)} = \cancel{T(N-2)} + 1$

$\cancel{T(N-2)} = \cancel{T(N-3)} + 1$

$\cancel{T(N-3)} = \cancel{T(N-4)} + 1$

$\cancel{T(N-4)} = \cancel{T(N-5)} + 1$

$\cancel{T(1)} = 1$

$T(n) = n$

$a^n$

$a^{n-1}$

n/2

$T(n) = k \;\; \log_2 n$

$\frac{N}{2^K} = 1$

$2^K = N$

$K = \log_2 N$

$T(n) = \cancel{T(N/2)} + 1$

$\cancel{T(N/2)} = \cancel{T(N/2^2)} + 1$

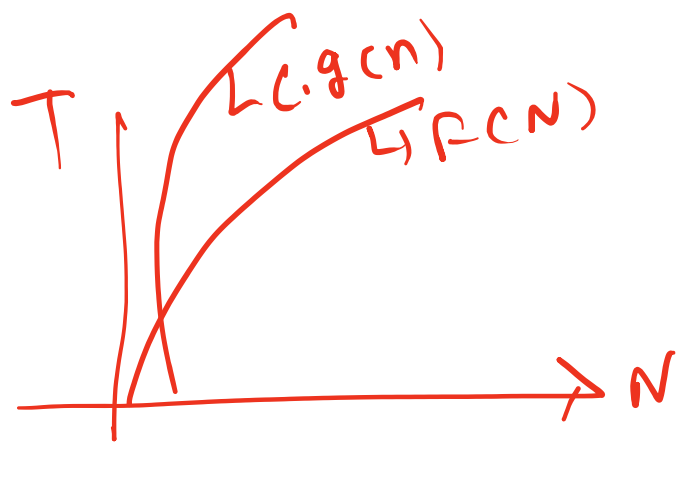
$\cancel{T(N/2^2)} = \cancel{T(N/2^3)} + 1$

$\cancel{T(N/2^3)} = \cancel{T(N/2^4)} + 1$

$\cancel{T(N/2^4)} = \cancel{T(N/2^5)} + 1$

$\cancel{T(N/2^K)} = 1$

```
public static int fib(int n) {
    if (n == 0 || n == 1) {
        return n;
    }
    int f1 = fib(n - 1);
    int f2 = fib(n - 2);
    return f1 + f2;
}
```



$\boxed{T(N) = \cancel{T(N-1)} + T(N-2) + 1}$

$\cancel{T(N-1)} = T(n-2) + T(N-3) + 1$

$\cancel{T(N-2)} = \cancel{T(N-3)} + T(n-4) + 1$

$\cancel{T(n-3)} = \cancel{T(n-4)} + T(n-5) + 1$

$\cancel{T(n-4)} = \cancel{T(n-5)} + T(n-6) + 1$

$\cancel{T(1)} = 1$

$f(N) \le c \, g(N) \rightarrow O(g(n))$

$T(N-1) + T(N-2) + 1 \le T(N-1) + T(n-1) + 1$

$T(n) = T(n-1) + T(n-1) + 1$

$T(n) = 2\,T(n-1) + 1$

$2\,\cancel{T(n-1)} = 2^2\,\cancel{T(n-2)} + 1 \times 2$

$2^2\,T(n-2) = 2^3\,\cancel{T(n-3)} + 1 \times 2^2$

$2^3\,T(n-3) = 2^4\,T(n-4) + 1 \times 2^3$

$2^4\,\cancel{T(n-4)} = 2^5\,T(n-5) + 1 \times 2^4$

$2^{N-1}\,\cancel{T(1)} = 1 \times 2^{N-1}$

$(n-1)$

T.C $2^N$

$T(n) = 2^0 + 2 + 2^2 + 2^3 + 2^4 \dots 2^{N-1} = 2^N - 1$

$\frac{2^0(2^N - 1)}{2-1}$

```
public static int[] Sort(int[] arr, int si, int ei) {
    if (si == ei) {
        int[] a = new int[1];
        a[0] = arr[si];
        return a;
    }
    int mid = (si + ei) / 2;
    int[] f = Sort(arr, si, mid);
    int[] s = Sort(arr, mid + 1, ei);
    return Merge_Two_Array(f, s);

}
```

n/2

n/2

$\frac{N}{2^K} = 1$

$2^K = N$

$K = \log_2 N$

$T(n) = KN$

$= N\log(n)$

$T(N) = 2\,\cancel{T(N/2)} + N$

$2\,\cancel{T(N/2)} = 2^2\,\cancel{T(N/2^2)} + \frac{N}{2} \times 2$

$2^2\,\cancel{T(N/2^2)} = 2^3\,\cancel{T(N/2^3)} + \frac{N}{2^2} \times 2^2$

$2^3\,\cancel{T(N/2^3)} = 2^4\,\cancel{T(N/2^4)} + \frac{N}{2^3} \times 2^3$

$2^4\,\cancel{T(N/2^4)} = 2^5\,\cancel{T(N/2^5)} + \frac{N}{2^4} \times 2^4$

$2^K\,T(N/2^K) = \frac{N}{2^K} \times 2^K$

Quick Sort

$\frac{n \times (n-1)}{2}$

$n^2$

$4 + 3 + 2 + 1$

$n-1 + n-2 + n-3 \dots 1$

nlog(n)

Random

[1 2 3 4 5]

⑤

1 2 3 4

4

1 2 3

3

1 2

②

[5 7 2 ③ 8 1 ④]

5 7 2 4 8 1 ③

1 2 3 4 5

1 5 4 3 ②

$3^8$

$3^4 \times 3^4 = 3^8$

ans = pow(a, n/2)

ans × ans

$3^{n/2} \times 3^{n/2}$

$3^7$

$3^3 \times 3^3 = 3^6$ Shorti

odd (ans × ans × a) odd

ans × ans × a